2 Technical Background

2.1 The Web PKI (Public Key Infrastructure)

Web PKI establishes internet trust by binding domain identities to cryptographic key pairs through X.509 digital certificates. Trusted Certificate Authorities (CAs) issue these certificates, which clients authenticate during a TLS handshake. However, the architecture relies on an additive trust model where any globally trusted CA can issue a valid certificate for any domain. A single CA compromise or misconfiguration introduces critical security vulnerabilities across the entire system. The following sections explain the structure of X.509 certificates, their issuance, existing security constraints, and the path validation process executed by client engines during a TLS handshake.

2.1.1 Certificates

An X.509 digital certificate acts as a cryptographic binding between a public key and an identity [1]. While server (leaf), intermediate, and root certificates share the same underlying data structure, their fields, constraints, and operational roles differ significantly.

Core Structure of an X.509 v3 Certificate

According to RFC 5280, every X.509 certificate contains the following standard fields [1]:

Field	Description
Version	Specifies the X.509 version (v3 supports extensions).
Serial Number	Positive integer assigned by the issuing CA, unique among the certificates issued by that CA.
Signature Algorithm Identifier	The cryptographic algorithm used by the CA to sign the certificate (e.g., sha256WithRSAEncryption or ecdsa-with-SHA256).
Issuer	The entity (CA) that verified the information and digitally signed the certificate.
Validity Period	The time window during which the certificate is valid, defined by notBefore and notAfter timestamps.
Subject	The entity associated with the public key contained in the certificate.

Subject Public Key Info (SPKI)	Contains the actual public key (e.g., RSA or ECDSA public key parameters) and the algorithm identifier. The SPKI identifies the certified public key and is used in public-key pinning mechanisms [1], [7]. In this thesis, SPKI fingerprints are also considered as identifiers for implementing a client-side Domain-to-CA binding policy.
X.509 Extensions (v3)	Additional configuration data, including Key Usage, Extended Key Usage (EKU), Subject Alternative Name (SAN), and Basic Constraints [1].
Signature Value	The digital signature computed by the issuer over the certificate payload, ensuring integrity and authenticity [1].

Certificate Types

End-entity server certificates are rarely signed directly by universally recognized root CAs; certificate validation requires constructing and verifying a certification path [1]. A valid chain typically consists of an End-Entity (Leaf) Certificate, Intermediate CA Certificates and a Trust Anchor (Root CA) [1], [5].

While the data layout is uniform, certificate types are distinguished by their Issuer/Subject relationship, Basic Constraints within a certification path [1], [5]:

A. Server (Leaf) Certificate

This certificate is issued for the specific server hostname and presented by the remote server during connection establishment.

- Role: Deployed on web servers to authenticate the domain during a TLS handshake [2].
- Issuer vs. Subject: The Issuer is an Intermediate CA, whereas the Subject represents the server's organizational identity and domain name [1].
- Key Fields & Extensions:
    - Subject Alternative Name (SAN): Contains the specific domain names or wildcards the certificate is valid for (e.g., app.scone.cloud or *.scone.cloud) [1], [4]. Browsers reject certificates if the requested hostname does not match an entry in the SAN extension [4].
    - Basic Constraints: Set to CA = FALSE (or omitted), explicitly prohibiting this certificate from issuing subordinate certificates [1].
    - Extended Key Usage (EKU): Typically restricted to Server Authentication [1].

B. Intermediate CA Certificate

These certificates bridge the gap between the leaf certificate and the top-level root authority.

- Role: It acts as a bridge between a root anchor and end-entity leaf certificates so protecting root private keys from operational exposure [1], [5].
- Issuer vs. Subject: The Issuer is either a Root CA or a parent Intermediate CA. The Subject is the intermediate organizational authority [1].
- Key Fields & Extensions:
    - Basic Constraints: Set to CA = TRUE and also defines a pathLenConstraint restricting how many Intermediate CAs can follow [1].
    - Key Cert Sign & CRL Sign: The Key Usage extension permits this key to sign other certificates and Certificate Revocation Lists (CRLs) [1].
    - Cross-Signatures: May contain multiple parent links if cross-signed by different root authorities to maintain backward compatibility [1].

C. Root CA Certificate

A pre-installed, self-signed root certificate embedded in the client application's local trust store.

- Role: It is a top anchor of trust embedded directly into client trust stores [1], [5].
- Issuer vs. Subject: Root certificates are self-signed hence the Issuer and Subject fields are identical [1].
- Key Fields & Extensions:
    - Self-Signature: Signed using its own private key, verified by checking that the public key matches the signature verification [1].
    - Basic Constraints: Set to CA = TRUE with broad trust permissions [1].
    - Trust Store Presence: Unlike leaves and intermediates which are transmitted over the network during a TLS handshake, root certificates must be pre-installed in the client application's local trust store [1], [2], [5].

2.1.2 Certificate Issuance

Certificate Authorities (CAs) do not generate server private keys or construct end-entity certificates unilaterally. Instead, certificate issuance is a dedicated administrative process done via manual enrollment or automated processes such as ACME (Automated Certificate Management Environment) [10]. The lifecycle of certificate issuance proceeds through several well-defined cryptographic steps [1], [10]:

Local Key Generation

The server administrator generates an asymmetric key pair (e.g., RSA or ECDSA) locally on the destination machine [10]. The corresponding private key remains strictly on the host and is never transmitted across the network [10].

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:2048 -out server.key
```

It produces a file named server.key containing both components of the asymmetric key pair (the private key and public key).

Certificate Signing Request (CSR) Construction

The host constructs a Certificate Signing Request (CSR) encoded in PKCS #10 format. The CSR contains the server's Subject Public Key Info (SPKI), the requested domain identity (specified within the Subject or Subject Alternative Name fields), and metadata, all digitally signed by the server's private key to prove possession [13].

```
openssl req -new  -key server.key -out domain.csr
-subj "/C=US/ST=California/L=San Francisco/O=Scone Group Inc./OU=IT/CN=app.scone.cloud" /
-addext "subjectAltName = DNS:app.scone.cloud, DNS:*.scone.cloud"
```

Verifying and Inspecting the Generated CSR

Before submitting the CSR to a Certificate Authority, verify its contents and cryptographic integrity locally using:

```
*openssl req -in domain.csr -noout -text -verify*
```

It decodes and renders the ASN.1 binary payload into human-readable text (Subject DN, SPKI public key parameters, and SAN extensions) and checks the self-signature on the CSR to mathematically prove that the private key matches the public key embedded inside the request.

Domain Validation (DV)

Upon receiving the CSR, the CA executes domain validation procedures (e.g., DNS challenge records, HTTP challenge endpoints) to confirm that the requesting server has administrative control over the target domain [10]. In a DNS challenge, the CA asks the domain owner to publish a unique cryptographic token as a TXT record in their DNS settings. CA verifies by querying the domain's name servers. HTTP challenge asks the owner to upload a temporary file to a specific URL path on their web server so the CA can validate control by downloading it over the internet.

Signing and Delivery

Once domain validation succeeds, the Certificate Authority (CA) constructs an X.509 end-entity certificate containing the server's Subject Public Key Info (SPKI), domain identity, validity period, issuer identity, and relevant extensions [1]. The domain identity is typically specified in the Subject Alternative Name (SAN) extension, which lists the DNS names for which the certificate is valid. The server generates and retains its own private key, while the CA uses the public key submitted in the Certificate Signing Request (CSR) to construct the certificate.

The CA assembles the To-Be-Signed Certificate (TBSCertificate), which contains the certificate's identifying information, public key, and extensions. It then signs the DER-encoded TBSCertificate using its private key, producing a digital signature that protects the integrity of the certificate's contents. The X.509 certificate consists of the TBSCertificate, the signature algorithm identifier, and the signature value [1].

After issuance, the CA provides the leaf certificate and necessary intermediate CA certificates to the server. During a TLS connection, the server presents this certificate chain to the client [2].

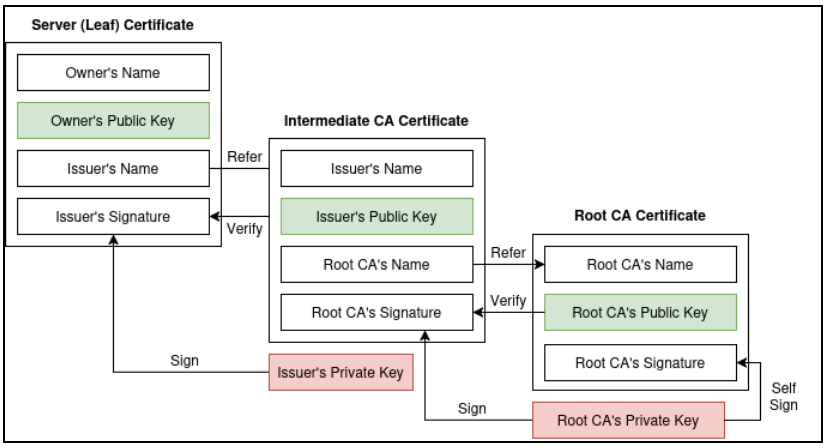


Figure 1: X.509 Certificate Chain Hierarchy and Signing Structure.

The Web Public Key Infrastructure relies on a hierarchical chain of trust that delegates cryptographic authority downward from a self-signed root Certificate Authority (CA) through intermediate subordinate CAs to an end-entity leaf certificate [5]. In this structure, each issuer signs the downstream subject's certificate with its private key, establishing an unbroken validation path where every intermediate and leaf certificate's Issuer field directly matches its parent's Subject field [5]

2.2 Certificate Authority Constraints and Existing Mitigation Approaches

The security of the Web Public Key Infrastructure (PKI) depends on the integrity of Certificate Authorities (CAs) and the correct enforcement of certificate issuance and validation policies. However, vulnerabilities in CA infrastructure, domain-validation procedures, cryptographic key generation or delegated certificate issuance can enable attackers to obtain unauthorized certificates for legitimate domains. Various mitigation mechanisms address these risks at different stages of the certificate, including restrictions on certificate authority, domain namespace constraints, pre-issuance authorization, post-issuance auditing, and certificate revocation. This section examines common CA compromise scenarios and evaluates the capabilities and limitations of existing mitigation approaches

2.2.1 CA Compromise Scenarios

Within this ecosystem, unauthorized or fraudulent certificate issuance typically exists through several distinct threat vectors:

CA Private Key Theft

If an attacker compromises a CA's internal network and extracts intermediate or root signing keys from Hardware Security Modules (HSMs) or operational servers, they can generate trusted certificates for any domains without the domain owner's knowledge or consent [8].

Domain Validation (DV) Protocol Exploits

Attackers can circumvent automated issuance controls by exploiting flaws in validation mechanisms. Using techniques such as Border Gateway Protocol (BGP) hijacking, DNS cache poisoning, or routing manipulation, an adversary can redirect a CA's challenge probes to an attacker-controlled server, falsely satisfying domain validation requirements [8], [10].

Compromised Intermediate CAs

Commercial CAs frequently delegate issuance authority to third-party resellers or subordinate intermediate entities. Weaker security hygiene in secondary partner organizations provides adversaries with secondary entry points to acquire unauthorized certificates that inherit global browser trust [8].

Weak Cryptography Key

Certificate Authorities occasionally suffer from software flaws or insufficient entropy in Random Number Generators (RNGs) used during key generation ceremonies. If a CA generates private keys with low entropy or flawed wrapping algorithms, an adversary can brute-force those private keys, allowing them to forge valid certificates from legitimate issuance..
A clear example occurred in October 2026, when a security researcher demonstrated full recovery of the private key material for eight eMudhra intermediate CA certificates (six TLS intermediates and two code-signing intermediates).

The incident demonstrates how flaws in CA key generation can compromise the security of the Web PKI. According to Mozilla Bug 2083449, private keys associated with eight eMudhra intermediate certificates were compromised because of a flaw in the key-generation process, making the private keys recoverable [11].
In response to the incident, Mozilla documented measures to block the affected intermediate certificates through OneCRL, its certificate revocation mechanism [11]. This incident highlights the importance of secure cryptographic key generation and the limitations of relying solely on the existing CA trust model. If an intermediate CA's private key is compromised, an attacker may be able to issue fraudulent certificates for domains within the scope of that CA's authority. The incident also motivates additional client-side protections, such as domain-to-CA binding, which could restrict the CAs accepted for a particular domain even when a certificate chains to an otherwise trusted root.

2.2.2 Existing Mitigation Approaches

The Public Key Infrastructure (PKI) employs several structural mechanisms to restrict certificate issuance and delimit the namespaces, cryptographic capabilities for which certificates may be utilized. These mechanisms function as mitigation approaches across distinct phases of the certificate lifecycle: prior to issuance through domain-level authorization policies, via cryptographic constraints embedded directly within X.509 certificate extensions, through ecosystem-wide auditing, and during client-side certification-path validation.

X.509 Name Constraints

The X.509 v3 certificate defines the Name Constraints extension, which enables an issuing Certificate Authority to restrict the namespaces within which intermediate CA certificates may validate end-entity identity [1]. The extension defines two structural sets: permittedSubtrees, which explicitly specifies the allowed namespace subtrees, and excludedSubtrees, which explicitly prohibits specified namespace subtrees [1].

For example, an intermediate CA certificate may incorporate a Name Constraints extension having permittedSubtrees for DNS names falling under ".scone.cloud" [1]. During client-side certification-path validation, a path-building engine evaluates whether every subject identity and Subject Alternative Name (SAN) entry in downstream certificates conforms to these subtree rules [1]. If any intermediate certificate in the chain violates a constraint, the path validation engine rejects the certification path [1].

Name Constraints provide a cryptographically enforced mechanism for scoping subordinate CA authority within hierarchical PKI structures [1]. However, Name Constraints do not provide a general mechanism for establishing domain-to-CA authorization rules within client browsers. Because they are encoded directly within intermediate CA certificates and enforced strictly as part of path validation, they rely entirely on the presence of constraints within that specific chain.

Name Constraints do not require a client to accept certificates for a given domain exclusively from a designated CA if an alternative, independently trusted certification path to a different trust anchor remains available [5], [8].

Basic Constraints and Path Length Restrictions

RFC 5280 defines the Basic Constraints extension to manage certificate hierarchies. This extension determines a certificate's role and limits how far down subordinate certificates can be issued [1]. It includes two primary components:

- CA Flag: It indicates whether the certified public key is authorized to verify digital signatures on subordinate certificates (cA = TRUE) or whether the certificate represents an end-entity leaf node (cA = FALSE) [1].
- pathLenConstraint Field: An optional integer present only when CA = TRUE that specifies the maximum number of intermediate CA certificates that may follow this certificate in a valid certification path [1].

By enforcing pathLenConstraint, an issuing authority restricts an intermediate from further delegating CA signing authority beyond an intended depth, limiting the operational blast radius of compromised intermediate keys [1], [5].

Key Usage and Extended Key Usage (EKU) Extensions

RFC 5280 uses the Key Usage and Extended Key Usage extensions to restrict how cryptographic keys can be used. These extensions prevent keys from being repurposed for unauthorized or incompatible tasks [1]:

- Key Usage: it is a bit string telling the precise cryptographic operations for which the public key may be applied. Intermediate CA certificates are required to set bits for keyCertSign permitting the verification of signatures on subordinate certificates and cRLSign permitting signature verification on revocation lists, but end-entity leaf certificates omit these permissions [1].
- Extended Key Usage (EKU): It indicates specific purpose Object Identifiers (OIDs) showing the application scenarios for which the certified key pair is valid. For instance, HTTP servers present certificates asserting id-kp-serverAuth (TLS Server Authentication). Client validation engines evaluate EKU assertions to ensure that specialized certificates such as S/MIME email signing (id-kp-emailProtection) or code signing (id-kp-codeSigning) are rejected if presented during a TLS handshake [1].

DNS Certification Authority Authorization (CAA)

CAA mechanism allows domain administrators to declare policy directly within the Domain Name System [3]. Domain owners publish CAA Records specifying which CAs are authorized to issue digital certificates for their specific hostnames or wildcards [3].

A typical CAA record set declared in a DNS zone file consists of flags, tags, and values structured as follows:

```
example.com.  IN  CAA  0 issue "letsencrypt.org"
example.com.  IN  CAA  0 issuewild ";"
example.com.  IN  CAA  0 iodef "mailto:security@example.com"
```

It contains an 8-bit integer field (e.g., 0 for non-critical, or 128 for critical, which instructs CAs to abort issuance if they do not recognize a tag). Tag specifies the authorization directive, such as issue (authorizing single or multi-domain certificates), issuewild (controlling wildcard certificate issuance, where ";" explicitly forbids all wildcard generation), or iodef (providing an endpoint for reporting policy violations). It also has the domain name of the authorized Certificate Authority (e.g., "letsencrypt.org").

However, CAA operates primarily as a pre-issuance constraint on CA behavior [3]. It does not independently alter client-side verification during a TLS handshake. A web browser does not check DNS CAA records when evaluating a presented certificate chain, nor does standard TLS validation reject a certificate solely because it was issued in violation of CAA policy [3]. Consequently, CAA alone fails to provide client-side defense against misissuance, CA compromise, or unauthorized certificates presented through alternative trusted paths [3].

2.2.3 Ecosystem-Wide Mitigations

The Web PKI relies on post-issuance auditing and revocation mechanisms to mitigate CA compromise:

Certificate Transparency (CT)

CT introduces public, append-only cryptographic logs built on Merkle Hash Trees (RFC 6962) [6]. Certificate Authorities are required to submit every issued certificate to multiple independent logs upon issuance, receiving a Signed Certificate Timestamp (SCT) in return. Browsers enforce CT by verifying that valid SCTs are presented during the TLS handshake either embedded in the certificate, stapled via TLS extensions, or delivered via OCSP [6].

However, CT functions primarily as a reactive auditing and detection framework rather than a real-time prevention mechanism against active MitM attacks. While it alerts domain owners to rogue certificate generation, it does not stop a browser from trusting a misissued certificate before the log entry is processed[6]. Also, non-browser clients (such as command-line runtimes, custom scripts, and embedded libraries) frequently omit or fail to enforce CT requirements entirely.

Revocation

Certificate Revocation Lists are periodically published ASN.1-encoded lists signed by the CA containing the serial numbers of revoked certificates [1]. CRLs suffer from severe scalability bottlenecks as file sizes grow rapidly, creating significant bandwidth and parsing overhead for clients [9].
Revocation checking is notoriously unreliable due to soft-fail default behaviors. If an attacker blocks network traffic to an OCSP responder or delays CRL downloads, browsers typically bypass the check and allow the connection to succeed [8], [9]. Emergency blocklists managed by browser vendors (such as Firefox's OneCRL or Chrome's CRLSets) suffer from propagation latency and remain entirely unavailable to non-browser applications, containerized environments, and custom TLS libraries [8], [9].

Online Certificate Status Protocol (OCSP)

OSCP allows clients to query a CA's responder in real-time for the validity status of a specific certificate. However, OCSP introduces severe privacy leaks (exposing user browsing habits to CAs) and network latency [9]. To address performance issues, OCSP Stapling enables servers to fetch and present a time-stamped, CA-signed OCSP response directly inside the TLS handshake [9]. An OCSP response contains the real-time validity status of a specific certificate indicating whether it is valid, revoked, or unknown along with a timestamp and a digital signature from the CA to cryptographically verify its authenticity.

HTTP Public Key Pinning (HPKP - RFC 7469)

HPKP was an early client-side mechanism where a web server delivered expected Subject Public Key Info (SPKI) fingerprints via an HTTP header (Public-Key-Pins), telling the browser to cache these constraints for subsequent connections. HPKP suffered from extreme operational fragility. It introduced catastrophic Trust-On-First-Use (TOFU) vulnerabilities on the initial unpinned connection, where a MitM attacker could inject fraudulent pins [7], [8]. Operator misconfiguration or lost private keys resulted in permanent, unrecoverable domain lockouts (self-inflicted denial-of-service) [7]. It also exposed domains to "ransomware pinning," where an adversary performing a temporary interception could pin an unauthorized key and permanently lock out legitimate administrators [7]. These severe failure modes led to the formal deprecation of HPKP across major browsers.

2.3 TLS 1.3 Handshake

While pre-issuance policies and certificate extensions define how certificates are structured and constrained, their actual security guarantees depend entirely on how they are processed during connection establishment. The following section explains the Transport Layer Security (TLS) protocol, specifically the TLS 1.3 handshake, how certificate chains are transmitted, authenticated, and evaluated by client path validation engines in practice.

HTTPS uses TLS 1.3 (RFC 9846) to deliver the certificate chain and to prove that the server holds the private key matching its leaf certificate [2]. The authentication-relevant messages occur in the following sequence:

1. ClientHello

   The client offers supported protocol versions, cipher suites, an (EC)DHE key share, supported signature algorithms, and the Server Name Indication (SNI). This lets the server select handshake parameters and the appropriate certificate [2].

2. ServerHello

   The server responds with its selected parameters and key share. At this point, both sides derive the handshake encryption keys, meaning all subsequent handshake messages are fully encrypted [2].

3. Certificate

   The server transmits its certificate chain, starting with the leaf certificate followed by any required intermediate certificates, along with any per-certificate extensions. This delivers the cryptographic material from which the client constructs and validates a path [2], [4].

4. CertificateVerify

   The server provides a digital signature over the handshake transcript, created using the private key corresponding to the public key in the leaf certificate [2]. This cryptographically proves possession of the certified private key [2].

5. Finished

   Each side sends a Message Authentication Code (MAC) over the handshake transcript, confirming that both endpoints observed the exact same handshake messages and preventing tampering [2].

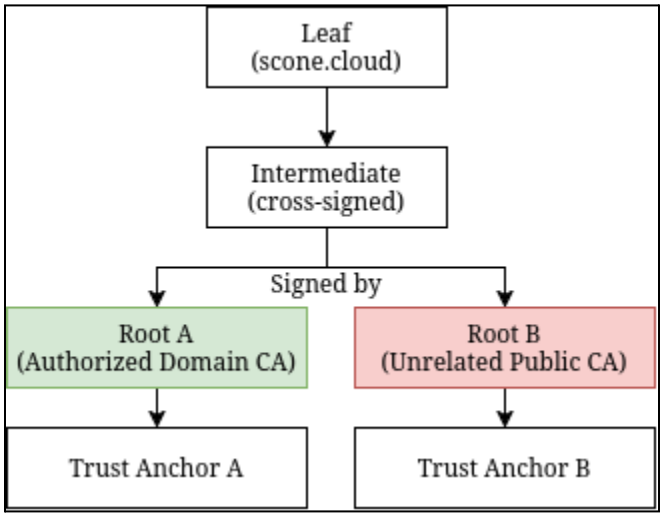
The handshake leaves several decisions to the client. First, the server's certificate authenticates a signature rather than the key exchange itself. Session keys are derived from an ephemeral (EC)DHE exchange, and the private key of the leaf certificate is used only to sign the handshake transcript in the CertificateVerify message. A valid chain alone therefore says nothing about which party completed the handshake; the client must verify that this signature is valid under the leaf's public key.

Second, In practice, certificate-chain construction can be complicated by incomplete or misordered chains and by differences in client-side handling of cached intermediates [2], [4]. Servers frequently send incomplete or misordered chains, and clients compensate with cached intermediates and Authority Information Access (AIA) fetching [4]. In passive measurement studies, 8.13% of valid chains could be validated only because of such caching [4].

Third, TLS delegates policy decisions: RFC 9846 treats detailed certificate validation as outside its scope and refers to the PKIX profile [1], [2]. Matching the certificate to the intended host is left to the application [2]. Also, TLS 1.3 encrypts the Certificate message, so passive observers can no longer read certificates directly from the wire [2].

2.3.1 Certification Path Validation

Certification-path validation follows the rules defined in RFC 5280, which specify how certificate chains and their constraints are evaluated. Hostname verification is a separate service-identity check, while revocation checking depends on the applicable validation policy and implementation [1], [2]. Path building discovers a sequence of certificates from the server's leaf to a trust anchor, while path validation verifies whether a given sequence satisfies the rules of RFC 5280 [1]. In practice, path building is complex due to cross-signed intermediates where the same leaf can chain to several anchors.



Standard rule is to accept if ANY path to ANY trusted anchor validates For a candidate path, RFC 5280 requires every certificate to pass a set of checks [1]:

Check	Verification
Signature	Each certificate is signed by the key of the next one up the path, ending at the trust anchor [1].
Validity period	The current time lies within notBefore/notAfter of every certificate [1].
Basic constraints	Every issuing certificate is a CA, and pathLenConstraint limits how many intermediates may follow [1].
Key usage	Issuing certificates assert keyCertSign [1].
Name constraints	Subject names and SANs in subordinate certificates fall within permitted subtrees and outside excluded subtrees [1].
Revocation status	No certificate on the path has been revoked [1].

2.3.2 Hostname Verification

A valid path alone proves only that some trusted CA vouched for a name. The client must also check that the certificate matches the host it intended to contact. For web services the presented identity is a DNS name in the Subject Alternative Name (SAN) extension, and the Common Name is no longer used as a fallback. Wildcards are restricted. In NSS, an asterisk may occur only in the leftmost label and matches exactly one label, so *.scone.cloud matches with thesis.scone.cloud but does not match master.thesis.scone.cloud [4].

Hostname verification prevents a certificate issued for a different domain from being accepted for the requested hostname. However, it does not establish that the issuing CA was authorized by the domain owner. A certificate issued by an unauthorized but globally trusted CA may still pass ordinary path validation and hostname verification if it contains the correct hostname and satisfies the other applicable checks [1], [2], [5]. It is also error-prone. Many applications and libraries skip or misuse hostname verification, leaving connections open to man-in-the-middle attacks [8].

Hostname verification confirms that the certificate names the right host. It does not confirm that the issuing CA was the one the domain owner authorized. A certificate for api.scone.cloud from any trusted CA passes both the path checks and the hostname check.

2.3.3 Revocation

Revocation determines whether a certificate that is otherwise valid has since been withdrawn, for example after key compromise. RFC 5280 defines Certificate Revocation Lists (CRLs) [1], and

RFC 6960 defines the Online Certificate Status Protocol (OCSP), which queries a responder for a single certificate [9]. In practice, revocation checking is applied unevenly. In a measurement study of the Web PKI, more than 8% of the certificates in their dataset were revoked, highlighting the practical challenges of certificate revocation, while many browsers do not check revocation at all, and mobile browsers never did in their tests [9]. Where checks are made, they are typically fail-open: if revocation data is unavailable, the certificate is accepted [8], [9]. Browser vendors also ship pre-compiled lists such as Chrome's CRLSets and Mozilla's OneCRL, but these cover only a small subset of revocations [9].

Revocation is reactive. It helps once a bad certificate has been identified and published. It cannot prevent a CA that is trusted for every domain from issuing a certificate that nobody has yet reported.

2.4 DomainTrustPolicy Bundles [12]

A DomainTrustPolicy bundle includes which Certificate Authority keys are authorized to vouch for a protected DNS subtree. Because this policy dictates trust enforcement, any unauthorized modification could enable an adversary to redirect trust to a malicious CA. To ensure bundle

integrity regardless of the delivery (HTTPS, file, or local disk), the policy bundle incorporates cryptographic signatures within its signatures field, enabling independent browser-side verification.

Roles

The policy issuer is typically an organization's security team which maintains an offline Ed25519 private key to sign policy iterations. Also, the client browser is provisioned solely with the corresponding public key. Each signature entry is structured as *{key_id, algorithm: "ed25519", signature}*, enabling the verification engine to resolve the appropriate public key.

2.4.1 Canonicalization (canonicalize_bundle - CFG-01)

Cryptographic signatures operate on raw byte sequences than abstract data structures. Because arbitrary JSON formatting, whitespaces, or key ordering yield disparate byte representations for identical semantics, the signing and verification engines must derive an identical byte stream through deterministic canonicalization. This process enforces strict serialization rules:

Exclusion of Signatures

The signatures field is stripped prior to canonicalization. This prevents self-referential signature validation and permits independent, multi-party signing.

Prohibition of Floating-Point Values

Floating-point numbers lack uniform textual representation across programming languages and architectures; policies are strictly restricted to integer values.

Deterministic Serialization

Object keys are recursively sorted at every level, serialized with compact separators ("," and ":"), and encoded in UTF-8 without extraneous whitespace. List preservation is maintained, as sequence order holds semantic weight.

For example, {"generation": 2, "environment": "dev"} serializes deterministically to {"environment":"dev","generation":2}. The parser explicitly rejects duplicate keys to eliminate discrepancies between the signer's serialization environment and the browser's parser.

2.4.2 Signing and Verification Workflow (CFG-02)

The cryptographic lifecycle proceeds through distinct phases:

Issuer Signing

The issuer canonicalizes the policy bundle bytes, computes an Ed25519 signature over the resulting byte string, and appends the signature entry to the bundle.

Browser Verification

Upon receipt, the browser reconstructs the canonical byte stream from the received bundle, queries the public key registry using the specified key_id, and cryptographically verifies the signature.

Any alteration to the signed content, including rules, authorized authorities, generation counters, or execution environments alters the canonical byte output and causes verification to fail. Signatures generated by unrecognized keys are classified as untrusted. A bundle is accepted only when a configurable signature validation threshold is satisfied; any supplementary invalid signatures are logged strictly as diagnostics to prevent denial-of-service via injected junk signatures.

2.4.3 Rule Compilation

The rules of a `DomainTrustPolicy` bundle are compiled into a *reverse-label trie*, a prefix tree whose edges are complete DNS labels ordered from the top-level domain downwards. The trie is built once, after the bundle has been authenticated and validated. All later lookups during certificate verification use this structure.

Construction

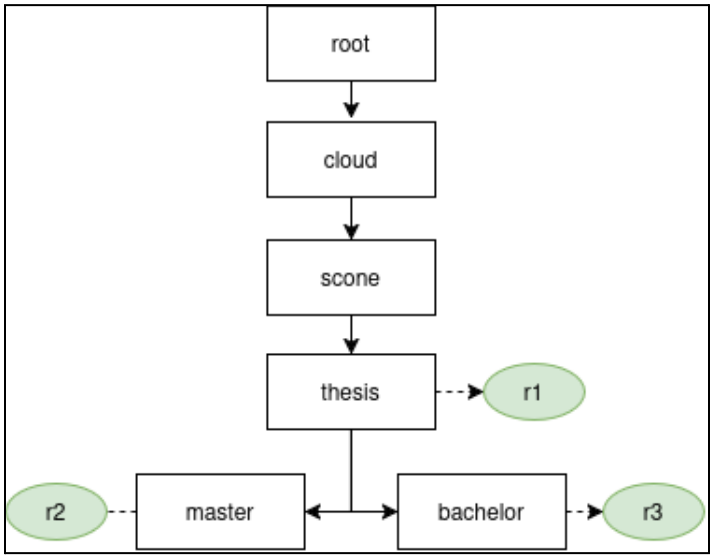
The rule's `dns_scope` is canonicalized: it is converted to lower case, any trailing dot is removed, and internationalized names are converted to their IDNA A-label form. IP literals are rejected. The canonical scope is split into labels and the label order is reversed. For example, `thesis.scone.cloud` becomes the path `cloud → scone → thesis`. The path is followed from the root, and missing nodes are created along the way. The rule is attached to the final node as a compiled rule. The compiled rule contains its identifier, enforcement mode, `include_subdomains` flag and a SHA-256 digest of the rule. It also contains the set of SPKI digests of its allowed authorities, which are resolved from the authority identifiers at compile time. If a node already carries a rule, two rules have the same specificity, and compilation fails. In that case the bundle is rejected as a whole.

2.4.4 Lookup

A hostname is canonicalized and traversed in the same reversed order. At each visited node that carries a rule, the rule is recorded as a candidate if it applies. A rule applies if the hostname ends at that node (an exact match), or if the rule has `include_subdomains` set. The traversal stops at the first missing label. If no rule applies, Chromium's default trust behavior is retained.

Rule	**DNS Scope**	**Allowed Authority**
r1	Thesis.scone.cloud (Include Subdomains= true)	university-ca
r2	Master.thesis.scone.cloud (Include Subdomains= false)	master-ca
r3	Bachelor.thesis.scone.cloud (Include Subdomains= true)	bachelor-ca

To structure these hierarchical scopes efficiently for runtime evaluation, rules are compiled into a reverse-label prefix tree where domain components are ordered from the top-level domain downwards. The resulting structural topology and corresponding rule assignments are organized as follows



Hostname	Matched Rule
master.thesis.scone.cloud	r2 (master-ca)
bachelor.thesis.scone.cloud	r3 (bachelor-ca)
avadhoot.bachelor.thesis.scone.cloud	r3 (bachelor-ca)
julius.master.thesis.scone.cloud	r1 (university-ca)
phd.thesis.scone.cloud	r1 (university-ca)
scone.cloud	No Rule (Fallback)
evil-thesis.scone.cloud	No Rule (Fallback)

A certificate for master.thesis.scone.cloud is therefore accepted only if its chain ends at master-ca. A certificate for any host under bachelor.thesis.scone.cloud must chain to bachelor-ca. Because the master and bachelor subtrees are separate branches, neither programme's CA can satisfy the other's rule. Since include_subdomains is false for r2, hosts below master.thesis.scone.cloud fall back to r1 and require university-ca. evil-thesis.scone.cloud matches no rule, even though its name ends in thesis.scone.cloud as a plain string, because the trie matches whole labels only.

2.5 Certificate Path Building and Validation in Firefox

Based on mozilla-central, Firefox 161 Nightly

2.5.1 Components

Component	Location	Role
Necko	netwerk/	Networking: DNS resolution, HTTP/1.1, HTTP/2 and HTTP/3 sessions.
PSM	security/manager/ssl	Personal Security Manager. Glue between Necko and NSS: socket setup, verification jobs, error handling and overrides, trust store loading.
CertVerifier	security/certverifier	Firefox's verification policy: EV and key-size attempts, Certificate Transparency, hostname check.
NSSCertDBTrustDomain	security/certverifier	Answers trust questions for the path builder: which certificates are anchors, where issuers come from, revocation, chain-level checks.
mozpkix	security/nss/lib/mozpkix	X.509 path builder and validator.

2.5.2 Trust sources

Firefox does not have a single trust store. NSSCertDBTrustDomain combines several sources

Source	Notes
Built-in roots	Mozilla Root Store in the nssckbi PKCS#11 module, loaded at startup. A chain ending here is a "built-in" chain: CT, distrust-after dates and static pinning apply.
NSS certificate DB	cert9.db in the profile. Holds user- or policy-imported certificates with trust bits, including explicit distrust.
Enterprise roots	Imported from the OS store when security.enterprise_roots.enabled is set; passed into the verifier as third-party roots.
cert_storage	OneCRL (revoked CAs), CRLite filters (revoked leaves), preloaded intermediates.
TLS handshake	Certificates sent by the server are issuer candidates, never anchors.

2.5.4 Path building in mozpkix

mozpkix builds a chain from the leaf upward with a recursive depth-first search. All trust knowledge comes from four TrustDomain callbacks implemented by NSSCertDBTrustDomain:

Algorithm

1. Check the current certificate on its own: validity period, basic constraints, key usage, extended key usage (serverAuth), certificate policies, and its trust level via GetCertTrust.
2. If it is a trust anchor, the chain is complete. IsChainValid is called with the whole chain. This happens before revocation and before the lower signatures are verified.
3. Otherwise, find issuers. FindIssuer returns candidates. For each candidate, PathBuildingStep::Check:
    - skips it if the subject name does not match, or if it would create a loop;
    - checks name constraints;
    - recurses to build the rest of the chain above the candidate;
    - verifies the child's signature with the candidate's key;
    - runs CheckRevocation for the child.
4. Backtrack on failure. If any step fails for a candidate, the error is recorded and the next candidate is tried. The search stops at the first fully valid chain.

Limits: at most 6 intermediates per chain, and a budget of 200,000 recursive calls per verification. If candidates fawhatil with different errors, the reported error becomes ERROR_UNKNOWN_ISSUER. Firefox does not fetch missing intermediates through AIA; it relies on what the server sends and on preloaded intermediates.

2.5.5 Validation checks

Check	Location	Behaviour
Signatures, key sizes, algorithms	mozpkix + trust domain	Each link verified. Weak RSA keys (below 2048 bits) are rejected for built-in chains.
Name constraints	mozpkix	Constraints in CA certificates, plus constraints NSS imposes on specific roots.
OneCRL	GetCertTrust	Revoked CA and intermediate certificates.
CRLite	CheckRevocation	Bloom-filter-based revocation for leaves. Only covers certificates logged in CT, so in practice public CAs.
OCSP and stapling	CheckRevocation	Stapled response first, then OCSP cache, then a live request if allowed. Soft-fail by default; hard-fail with security.OCSP.require. OCSP must-staple is enforced after the chain is built.
Key pinning	IsChainValid	Static pins for selected hosts; rejects chains without a pinned key, causing backtracking.
Distrust-after	IsChainValid	Built-in roots distrusted for certificates issued after a date.
Certificate Transparency	VerifyCertificateTransparencyPolicy	After the chain is built. Only enforced when the root is a built-in root; private CAs need no SCTs.
EV	VerifyCert	Tried first with the EV policy OID; falls back to a normal (DV) attempt.
Hostname	CheckCertHostname	Last step. Matches the requested name against the leaf's SAN entries; a certificate wildcard covers exactly one label.

2.6 References

[1] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk, "Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile," RFC 5280, IETF, May 2008.

[2] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," RFC 9846, IETF, 2026.
[3] P. Hallam-Baker, R. Stradling, and J. Hoffman-Andrews, "DNS Certification Authority Authorization (CAA) Resource Record," RFC 8659, IETF, Nov. 2019.

[4] D. Akhawe, J. Amann, M. Vallentin, and R. Sommer, "Here's My Cert, So Trust Me, Maybe? Understanding TLS Errors on the Web," in Proc. 22nd Int. Conf. World Wide Web (WWW), 2013.

[5] Z. Durumeric, J. Kasten, M. Bailey, and J. A. Halderman, "Analysis of the HTTPS Certificate Ecosystem," in Proc. 13th ACM Internet Measurement Conf. (IMC), 2013.

[6] B. Laurie, A. Langley, and E. Kasper, "Certificate Transparency," RFC 6962, IETF, Jun. 2013.

[7] C. Evans, C. Palmer, and R. Sleevi, "Public Key Pinning Extension for HTTP," RFC 7469, IETF, Apr. 2015.

[8] J. Clark and P. C. van Oorschot, "SoK: SSL and HTTPS: Revisiting Past Challenges and Evaluating Certificate Trust Model Enhancements," in Proc. IEEE Symp. Security and Privacy (S&P), 2013..

[9] Y. Liu, W. Tome, L. Zhang, D. Choffnes, D. Levin, B. Maggs, A. Mislove, A. Schulman, and C. Wilson, "An End-to-End Measurement of Certificate Revocation in the Web's PKI," in Proc. 15th ACM Internet Measurement Conf. (IMC), 2015.

[10] J. Aas, R. Barnes, B. Case, Z. Durumeric, P. Eckersley, A. Flores-López, J. A. Halderman, J. Hoffman-Andrews, J. Kasten, E. Rescorla, S. Schoen, and B. Warren, "Let's Encrypt: An Automated Certificate Authority to Encrypt the Entire Web," in Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS), 2019.

[11] Mozilla, "Bug 2083449, eMudhra: Private key compromise of Intermediate Certificates," Bugzilla, Oct. 2026. [Online]. Available: https://bugzilla.mozilla.org/show_bug.cgi?id=2083449

[12] chromium-custom-verifier-macos
https://github.com/SystemsEngineering1/chromium-custom-verifier-macos

[13] RFC 2986, PKCS #10: Certification Request Syntax Specification Version 1.7.